

API операционных систем

Введение в API Windows, файловый ввод-вывод

Подготовил
Старший преподаватель
МИЭМ НИУ ВШЭ
Морозов Владимир Игоревич

Основные отличия от классического программирования

Основные черты WinAPI

- Фундаментальные типы данных Си практически не используются. Они заменены на специфичные типы, названия которых, как правило, пишутся заглавными буквами (например, **BOOL** или **HANDLE**)
- Указатели, как правило, определены в виде отдельных типов данных (например, чтобы объявить указатель на тип данных **DWORD** используют запись **LPDWORD**, а не **DWORD ***)
- Большинство функций имеют две вариации: для работы со строками обычных символов (**char**) и UNICODE-строками (**wchar_t**). Эта особенность будет рассмотрена далее.
- Доступ к большинству объектов ядра осуществляется по единому идентификатору типа **HANDLE**

Основные черты WinAPI

- Многие функции Windows API имеют очень широкую область применения, а также обладают большой гибкостью, в связи с чем могут принимать множество различных аргументов, большая часть которых в наших задачах будет просто устанавливаться равной **NULL** или некоторым заранее определённым значениям по умолчанию
- Большинство функций WinAPI может быть подключено с помощью заголовочного файла `windows.h`. Подключение других заголовков требуется достаточно редко.

Строки в WinAPI

- Функции, работающие со строками в Windows API, имеют по две вариации: для работы с обычными (ASCII) и расширенными (UNICODE) строками
- Вариации функций для ASCII, как правило, имеют в конце названия букву **A**, а для UNICODE – букву **W**. Например, функция **CreateFile**, принимающая имя файла в виде ASCII-строки, называется **CreateFileA**, а её аналог, работающий с UNICODE строкой, – **CreateFileW**.
- Часто такие функции представлены в обобщенном виде (без последней буквы). В таком случае нужная вариация выбирается в зависимости от того, определены ли макросы **UNICODE** и **_UNICODE**

Обработка ошибок

Способы обработки ошибок в Windows

- Возвращаемым значением многих функций WinAPI является булево значение (тип **BOOL**), где **TRUE** означает успешное выполнение, а **FALSE** – возникновение ошибки. Необходимо иметь в виду, что данное соглашение выполняется не всегда. Для подробной информации по каждой функции необходимо обращаться к документации.
- Чтобы получить код последней возникшей ошибки, используется функция **GetLastError()**
- Для получения текстового описания ошибки можно воспользоваться универсальной функцией **FormatMessage**

Пример обработки ошибок в Windows

- В представленном листинге кода (на следующем слайде) показан пример обработки ошибки, возникающей при попытке открытия несуществующего файла с помощью функции **CreateFile**
- Результатом выполнения представленного кода станет сообщение *Не удается найти указанный файл.*
выведенное в стандартный поток вывода


```
setlocale(0, "");

HANDLE hFile = CreateFile("hello", GENERIC_READ, 0, NULL, OPEN_EXISTING, 0, NULL);

if (hFile == INVALID_HANDLE_VALUE)

{

    LPSTR message;

    DWORD dwMessageLen = FormatMessage(
        FORMAT_MESSAGE_FROM_SYSTEM | FORMAT_MESSAGE_ALLOCATE_BUFFER,
        NULL,
        GetLastError(),
        MAKELANGID(LANG_NEUTRAL, SUBLANG_DEFAULT),
        (LPSTR) &message,
        0,
        NULL);

    printf(message);

    HeapFree(GetProcessHeap(), 0, message);

    return 1;

}
```

Константы с номерами ошибок

- Как и в Linux, в WinAPI существует широкий набор констант, хранящих номера всех ошибок, которые могут произойти при использовании функций. К примеру, ошибка, произошедшая в предыдущем примере (при попытке открытия несуществующего файла), имеет номер 2, заданный в константе `ERROR_FILE_NOT_FOUND`.
- Полный список констант с ошибками можно найти в документации

Файловый ввод-вывод

Основные положения

- Дескриптор открытого файла представляется в WinAPI универсальным типом данных **HANDLE**, который может хранить идентификаторы большинства объектов ядра
- Для осуществления базовых операций файлового ввода-вывода в Windows используются 4 функции:
 - **CreateFile** – открывает файл, имя которого передано в качестве аргумента и возвращает дескриптор открытого файла
 - **ReadFile** – читает необходимое количество байтов из файла в буфер (массив байтов)
 - **WriteFile** – записывает необходимое количество байтов из буфера в файл
 - **CloseHandle** – прекращает доступ к любому объекту ядра (в т.ч. и к файлу) по идентификатору типа **HANDLE**
- Существует больше функций для работы с файлами, в рамках данного занятия рассматриваются только основные

Функция CreateFile

- **CreateFile** принимает семь аргументов:
 - **lpFileName** – имя открываемого файла
 - **dwDesiredAccess** – флаги типа доступа к файлу (на чтение или на запись)
 - **dwShareMode** – используется для открытия файла для совместного использования (с другими процессами, потоками или даже в рамках данного потока)
 - **lpSecurityAttributes** – указатель на структуру с параметрами безопасности для вновь создаваемого файла (будем устанавливать равным **NULL**)
 - **dwCreationDisposition** – флаги создания файла (указывают специфику создания нового файла или открытия существующего)
 - **dwFlagsAndAttributes** – дополнительные атрибуты для вновь создаваемого файла (будем устанавливать равным **FILE_ATTRIBUTE_NORMAL**)
 - **hTemplateFile** – дескриптор файла-шаблона, из которого берутся значения для флагов вновь создаваемого файла (будем устанавливать равным **NULL**)
- Возвращает **HANDLE** на открытый файл в случае успешного открытия и **INVALID_HANDLE_VALUE** в случае ошибки

Флаги типа доступа

- **CreateFile** может принимать два флага типа доступа:
 - **GENERIC_READ** – доступ на чтение
 - **GENERIC_WRITE** – доступ на запись
- Эти два флага могут быть объединены с помощью логического ИЛИ для открытия файла на чтение и запись

Флаги совместного использования

- **CreateFile** может принимать 4 флага совместного использования:
 - 0 – совместный доступ к файлу запрещён
 - **FILE_SHARE_DELETE** – разрешает совместный доступ для удаления и переименования
 - **FILE_SHARE_READ** – разрешает совместный доступ для чтения
 - **FILE_SHARE_WRITE** – разрешает совместный доступ для записи
- Последние три из этих флагов могут быть объединены с помощью логического ИЛИ

Флаги создания файла

- **CreateFile** может принимать 5 флагов создания файла:
 - **CREATE_NEW** – производится создание нового файла. Если файл с указанным именем уже существует, возвращается ошибка.
 - **CREATE_ALWAYS** – производится создание нового файла. Если файл с указанным именем уже существует, он перезаписывается.
 - **OPEN_EXISTING** – производится открытие существующего файла. Если файл с указанным именем не существует, возвращается ошибка.
 - **OPEN_ALWAYS** – производится открытие существующего файла. Если файл с указанным именем не существует, он создаётся.
 - **TRUNCATE_EXISTING** – производится открытие существующего файла. При открытии информация в файле стирается. Для использования этого флага должен быть установлен флаг типа доступа **GENERIC_WRITE**.
- Объединять данные флаги не следует

Функция ReadFile

- **ReadFile** принимает пять аргументов:
 - **hFile** – дескриптор открытого для чтения файла типа **HANDLE**
 - **lpBuffer** – указатель на буфер для считанной информации (массив байт), **память должна быть выделена заранее!**
 - **nNumberOfBytesToRead** – количество байт, которые необходимо считать
 - **lpNumberOfBytesRead** – указатель на переменную типа **DWORD**, в которую сохранится количество фактически считанных байт
 - **lpOverlapped** – указатель на структуру с параметрами асинхронного чтения (будем устанавливать равным **NULL**)
- Возвращает **TRUE** в случае успешного вызова и **FALSE** в случае ошибки
- Запрос количества байтов большего, чем размер выделенной под буфер памяти, приводит к неопределённому поведению и **должен избегаться**

Пример использования ReadFile

- В данном примере (на следующем слайде) из файла file.txt будет считана строка символов размером 5 байт.

```
HANDLE hFile = CreateFile("file.txt", GENERIC_READ, 0, NULL, OPEN_ALWAYS, FILE_ATTRIBUTE_NORMAL, NULL);

if (hFile == INVALID_HANDLE_VALUE)

{

    printf("Open error: %d\n", GetLastError());

    return 1;

}

LPSTR buf = malloc(sizeof(CHAR) * 6);

DWORD dwBytesRead = 0;

if (ReadFile(hFile, buf, 5, &dwBytesRead, NULL))

{

    printf("%s\n", buf);

}

else

{

    printf("Read error: %d\n", GetLastError());

    return 1;

}

CloseHandle(hFile);

free(buf);
```

Функция WriteFile

- **WriteFile** принимает пять аргументов:
 - **hFile** – дескриптор открытого для записи файла типа **HANDLE**
 - **lpBuffer** – указатель на буфер для записываемой в файл информации (массив байт)
 - **nNumberOfBytesToWrite** – количество байт, которые необходимо записать
 - **lpNumberOfBytesWritten** – указатель на переменную типа **DWORD**, в которую сохранится количество фактически записанных байт
 - **lpOverlapped** – указатель на структуру с параметрами асинхронной записи (будем устанавливать равным **NULL**)
- Возвращает **TRUE** в случае успешного вызова и **FALSE** в случае ошибки
- Запись количества байтов большего, чем размер выделенного буфера, приводит к неопределённому поведению и **должна избегаться**

Пример использования WriteFile

- В данном примере (на следующем слайде) в файл file.txt будет записана строка символов размером 5 байт.

```
HANDLE hFile = CreateFile("file.txt", GENERIC_WRITE, 0, NULL, OPEN_ALWAYS, FILE_ATTRIBUTE_NORMAL,
NULL);

if (hFile == INVALID_HANDLE_VALUE)

{

    printf("Open error: %d\n", GetLastError());

    return 1;

}

LPSTR message = "Hello";

DWORD dwBytesWritten = 0;


if (!WriteFile(hFile, message, 5, &dwBytesWritten, NULL))

{

    printf("Error: %d\n", GetLastError());

}

CloseHandle(hFile);
```

Запись в конец файла

- В отличие от Linux, чтобы вести запись в конец файла, в WinAPI недостаточно открыть файл со специальным флагом
- Для записи в конец файла перед вызовом **WriteFile** нужно вызвать специальную функцию **SetFilePointer**, устанавливающую позицию “курсора” в файле, следующим образом:

```
SetFilePointer(hFile, 0, NULL, FILE_END);
```

Консольный ввод-вывод

- Чтобы осуществлять ввод и вывод в консоль, можно воспользоваться функциями для файлового ввода-вывода, передав им дескриптор нужного псевдофайла (стандартного ввода, стандартного вывода или стандартной ошибки)
- Чтобы получить дескриптор псевдофайла, связанного с консолью, необходимо выполнить вызов функции **GetStdHandle** с одним из трёх аргументов:
 - **STD_INPUT_HANDLE** – для получения дескриптора стандартного ввода
 - **STD_OUTPUT_HANDLE** – для получения дескриптора стандартного вывода
 - **STD_ERROR_HANDLE** – для получения дескриптора стандартной ошибки
- Закрывать полученный дескриптор посредством **CloseHandle** не следует

Консольный ввод-вывод

- Для осуществления консольного ввода-вывода также существует специальная пара функций, использование которых предпочтительно:
 - `ConsoleRead` – для чтения данных из консоли
 - `ConsoleWrite` – для вывода данных на консоль
- Аргументы этих функций аналогичны аргументам функций `ReadFile` и `WriteFile` соответственно

Пример консольного ввода-вывода

- В листинге кода, представленном ниже, производится вывод в консоль сообщения “Hello”.

```
LPSTR message = "Hello";

DWORD dwBytesWritten = 0;

HANDLE hConsole = GetStdHandle(STD_OUTPUT_HANDLE);

if (hConsole == INVALID_HANDLE_VALUE)

{

    return 1;

}

if(!WriteConsole(hConsole, message, 5, &dwBytesWritten, NULL))

{

    return 1;

}
```

Документация

- Наиболее подробную документацию по всем функциям файлового ввода-вывода WinAPI можно найти на сайте <https://docs.microsoft.com/en-us/windows/win32/api/fileapi/>

Полезная литература

- Джонсон М. Харт. Системное программирование в среде Windows. – Книга, на основании которой сделана данная презентация. Для закрепления материала рекомендуется глава 2.